Password Policy Validation Engine

1. Problem Statement

The Password Policy Validation Engine is a Python program designed to check whether a password satisfies a set of security requirements. The program checks the password length, the presence of uppercase and lowercase letters, digits, special characters, repeated characters, and prohibited text.

The program displays whether the password is valid or invalid and identifies the requirements that are not satisfied. This helps users create stronger passwords and improves basic password security.

2. Objectives

1. To validate passwords according to predefined security rules.
2. To check whether the password has the required minimum length.
3. To verify the presence of uppercase letters, lowercase letters, digits, and special characters.
4. To detect excessive repeated characters.
5. To identify prohibited text in a password.
6. To display the validation results clearly.

3. Assumptions

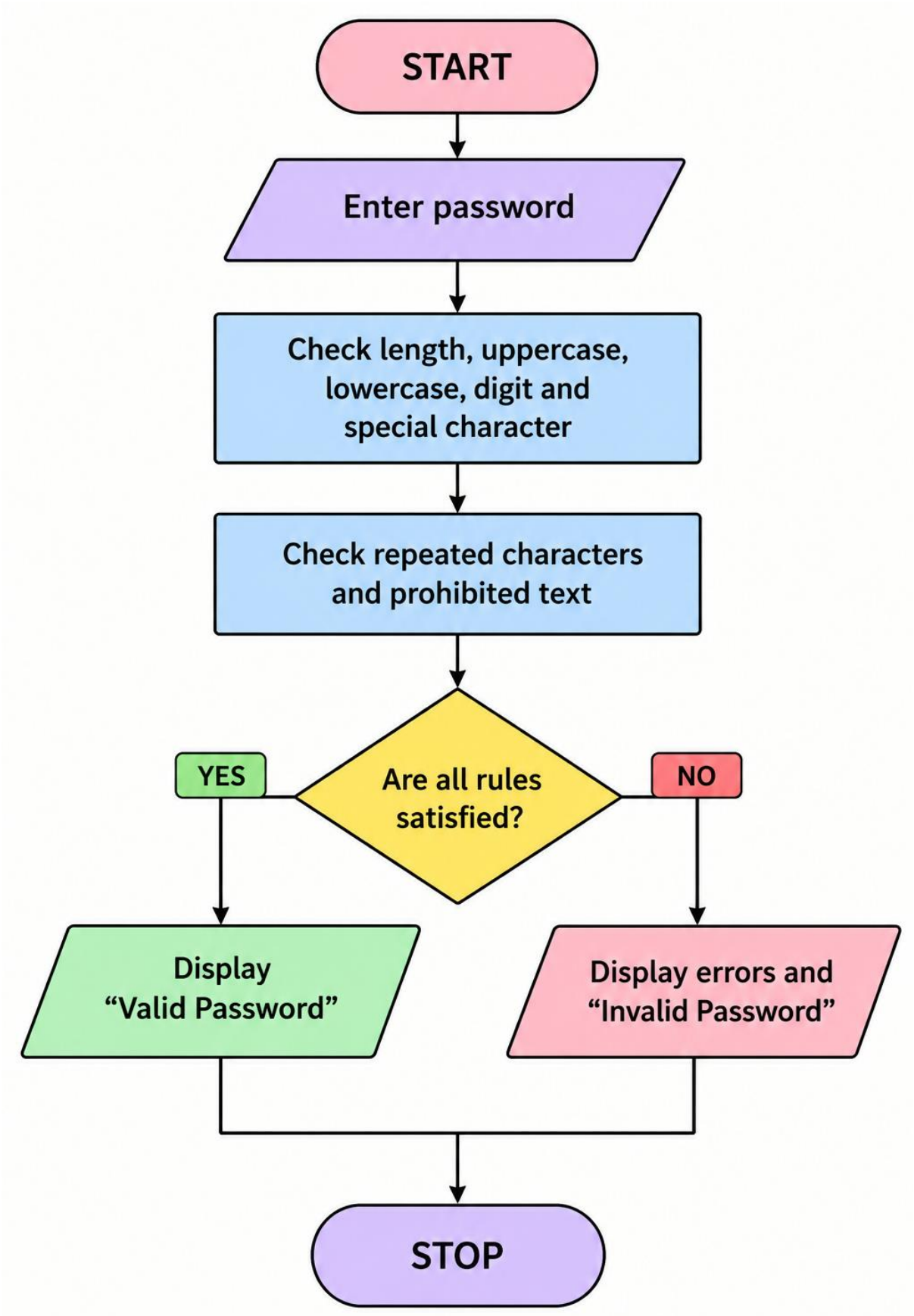
1. The minimum password length is 8 characters.
2. The password must contain at least one uppercase letter and one lowercase letter.
3. The password must contain at least one digit.
4. The password must contain at least one special character, such as @, #, $, or !.
5. Three or more consecutive identical characters are not allowed.
6. The password must not contain the prohibited text password or admin, regardless of capitalization.

7. The program accepts a password entered by the user.
8. The password is considered valid only when all the rules are satisfied.

4. Algorithm

1. Start the program.
2. Read a password from the user.
3. Initialize an empty list to store validation errors.
4. Check whether the password contains at least 8 characters.
5. Check for at least one uppercase letter.
6. Check for at least one lowercase letter.
7. Check for at least one digit.
8. Check for at least one special character.
9. Use a loop to detect three or more consecutive identical characters.
10. Convert the password to lowercase and check for prohibited text.
11. If any rule fails, add the corresponding error message to the list.
12. If the list is empty, display that the password is valid.
13. Otherwise, display that the password is invalid and show the failed requirements.
14. Stop the program.

5. Flowchart


START
Enter password
Check length, uppercase, lowercase, digit and special character
Check repeated characters and prohibited text
Are all rules satisfied?
YES
NO
Display "Valid Password"
Display errors and "Invalid Password"
STOP

6. Pseudocode

```
START

INPUT password
SET errors = empty list

IF length of password < 8
    ADD "Minimum length is 8 characters" to errors

IF password has no uppercase letter
    ADD "At least one uppercase letter is required" to errors

IF password has no lowercase letter
    ADD "At least one lowercase letter is required" to errors

IF password has no digit
    ADD "At least one digit is required" to errors

IF password has no special character
    ADD "At least one special character is required" to errors

FOR each position in password
    IF three consecutive characters are identical
        ADD "Repeated characters are not allowed" to errors
```

```
        BREAK loop

CONVERT password to lowercase

IF password contains "password" or "admin"
    ADD "Prohibited text detected" to errors

IF errors is empty
    DISPLAY "Valid Password"
ELSE
    DISPLAY "Invalid Password"
    DISPLAY all errors

STOP
```

7. Python Actual Code

```
password = input("Enter your password: ")

errors = []

# Check minimum length
if len(password) < 8:
    errors.append("Password must contain at least 8 characters.")
```

```
# Check uppercase letter
if not any(ch.isupper() for ch in password):
    errors.append("Password must contain an uppercase letter.")

# Check lowercase letter
if not any(ch.islower() for ch in password):
    errors.append("Password must contain a lowercase letter.")

# Check digit
if not any(ch.isdigit() for ch in password):
    errors.append("Password must contain a digit.")

# Check special character
if not any(not ch.isalnum() for ch in password):
    errors.append("Password must contain a special character.")

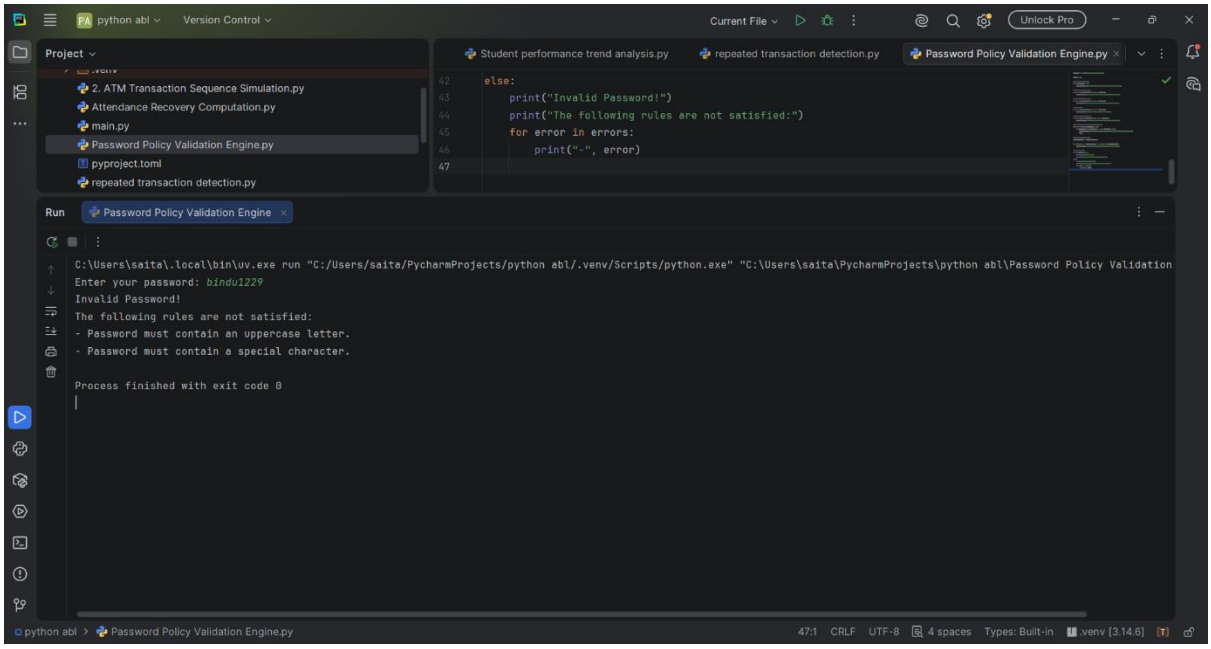
# Check consecutive repeated characters
for i in range(len(password) - 2):
    if password[i] == password[i + 1] == password[i + 2]:
        errors.append("Three consecutive identical characters are not allowed.")
        break

# Check prohibited text
lower_password = password.lower()
```

```
if "password" in lower_password or "admin" in lower_password:
    errors.append("Password must not contain prohibited text.")


# Display result
if len(errors) == 0:
    print("Valid Password!")
    print("All password rules are satisfied.")
else:
    print("Invalid Password!")
    print("The following rules are not satisfied:")
    for error in errors:
        print("-", error)
```

8. Output



9. Explanation of the Code

1. Input: input() accepts the password entered by the user.

2. Error list: The errors list stores messages for rules that fail.
3. Length check: len(password) checks the number of characters.
4. Character checks: any(), isupper(), islower(), and isdigit() check the required character categories.
5. Special character check: isalnum() identifies letters and digits; not ch.isalnum() detects a character outside those categories.
6. Repeated character check: A for loop compares three consecutive characters to identify repeated sequences.
7. Prohibited text check: lower() converts the password to lowercase so that words such as Admin and PASSWORD can be detected regardless of capitalization.
8. Final result: The program displays a success message if there are no errors; otherwise, it prints every failed requirement.

10. Conclusion

The Password Policy Validation Engine successfully checks whether a password follows predefined security rules. It validates length, character categories, repeated characters, and prohibited text using Python strings, loops, conditions, and string operations. The program provides clear feedback to users and helps them understand how to create stronger passwords. This project demonstrates how basic Python programming concepts can be applied to a practical password validation problem.